

《100天风控专家》

Lift提升度指标计算

出品：东哥起飞

解锁风控课程



关注我的公众号





1. 什么是Lift?

Lift (提升度) 是评估模型或者规则是否有效的一个度量指标, **它表示模型或规则对于目标用户的预测能力相对于随机预测的提升程度 (倍数)。**

Lift提升度最初是应用在营销场景中的, 举个例子, 比如现在我们给100个客户营销, 其中有10个响应, 那么响应率就是10%, 这是我们不使用任何手段的条件下随机营销的结果。对于这个结果公司可能不满意, 于是针对该场景开发了营销响应模型, 通过模型的预测结果进行营销。现在使用模型预测结果对其它条件相同的100个客户营销, 发现有30个客户响应, 那么响应率就是30%。从10%到30%, 响应率指标提升了3倍, 这个就是Lift提升度, **用来衡量模型相对于随机情况下的效用, 倍数越高代表提升越多, 效果越好。**

Lift提升度的指标不仅限于营销场景, 也可以应用在风控场景中。本质是一样的, 只是场景不同。比如现在我们有样本数量有5000个, 其中坏客户数500个, 坏客户浓度为10%。如果我们从中“随机地”选择500个通过, 那么则有差不多 $500 \times 10\% = 50$ 个坏客户。但如果我们使用了“风控模型或规则”对样本预测并排序, 只选择结果最好的500个通过, 假设这500个客户全部用信并且最终坏账的只有10户。那么和随机选择相比, 模型或规则的应用就是有效的。



扫码进专属群





2. Lift计算逻辑

Lift = 分箱下的区间坏账率/总样本坏账率。 计算步骤为:

- 1) 首先对被计算对象（模型或规则）进行分箱，比如10等分或20等分的等频分箱
- 2) 统计各箱体下的样本数量、坏样本数量、区间坏账率；**区间坏账率=每箱坏样本数/每箱总样本数**；
- 3) 计算“区间坏账率/总样本坏账率”

xgb_score	sum	bad_count	good_count	badrate	lift
(205.388, 448.898]	1000	150	850	15.00%	3.00
(448.898, 483.828]	1000	100	900	10.00%	2.00
(483.828, 508.842]	1000	70	930	7.00%	1.40
(508.842, 530.35]	1001	50	951	5.00%	1.00
(530.35, 550.93]	1001	45	956	4.50%	0.90
(550.93, 570.996]	998	35	963	3.51%	0.70
(570.996, 592.425]	1000	20	980	2.00%	0.40
(592.425, 618.716]	1000	15	985	1.50%	0.30
(618.716, 654.116]	1000	10	990	1.00%	0.20
(654.116, 852.11]	1000	5	995	0.50%	0.10
all	10000	500	9500	5.00%	1.00



扫码进专属群





2. Lift计算逻辑

Lift = 累计坏样本占比/累计总样本占比。 计算步骤为：

- 1) 首先对被计算对象（模型或规则）进行分箱，比如10等分的等频分箱
- 2) 统计各箱体下的样本数量、坏样本数量、累计坏样本占比、累计总样本占比；**坏样本占比=每箱坏样本数/总的坏样本数；总样本占比=每箱样本数/总的样本数。**
- 3) 计算“累计坏样本占比/累计总样本占比”

xgb_score	sum	bad_count	good_count	badpct	allpct	cum_badpct	cum_allpct	cum_lift
(205.388, 448.898]	1000	150	850	30.00%	10.00%	30.00%	10.00%	3.00
(448.898, 483.828]	1000	100	900	20.00%	10.00%	50.00%	20.00%	2.50
(483.828, 508.842]	1000	70	930	14.00%	10.00%	64.00%	30.00%	2.13
(508.842, 530.35]	1001	50	951	10.00%	10.01%	74.00%	40.01%	1.85
(530.35, 550.93]	1001	45	956	9.00%	10.01%	83.00%	50.02%	1.66
(550.93, 570.996]	998	35	963	7.00%	9.98%	90.00%	60.00%	1.50
(570.996, 592.425]	1000	20	980	4.00%	10.00%	94.00%	70.00%	1.34
(592.425, 618.716]	1000	15	985	3.00%	10.00%	97.00%	80.00%	1.21
(618.716, 654.116]	1000	10	990	2.00%	10.00%	99.00%	90.00%	1.10
(654.116, 852.11]	1000	5	995	1.00%	10.00%	100.00%	100.00%	1.00
all	10000	500	9500	100.00%	100.00%			



扫码进专属群



3. Lift应用和评估标准

应用对象

Lift提升度可以应用在**模型、规则**上。模型的输出结果为一个概率或者概率转化后的分数，可以作为风险分层使用，也可以作为强规则使用。当模型作为规则使用时，模型和规则本质一样，此时Lift用于评估相较随机情况下的提升倍数，值越高代表提升越高，效果越好。同时，Lift值的大小也可用来作为制定阈值cutoff的标准。

评估标准

Lift指标的数值以“1”为界线，代表的含义列举如下。那Lift指标具体要达到多少比较合适？这个需要结合具体的业务成本等指标综合分析确定。**在信贷业务场景中，一般情况下，如果大于3说明有较好的提升效果。**

与1比较	含义
Lift值大于1	表示预测能力优于随机预测，且值越高预测能力越强。
Lift值等于1	表示预测能力与随机预测相同，即没有提供额外的预测能力，和盲猜一样。
Lift值小于1	表示预测能力不如随机预测，即预测准确率比盲猜都低，起反作用。



扫码进专属群





4. Lift计算的Python代码



扫码进专属群



```
# 等频分箱
df_bin=pd.qcut(df['xgb_score'],10)
group_all=df['target'].groupby(df_bin).count()
group_bad=df['target'].groupby(df_bin).sum()
group_good=group_all-group_bad
var_bin=pd.DataFrame()
var_bin['sum']=group_all
var_bin['bad_count']=group_bad
var_bin['good_count']=group_good

# 第一种Lift计算方式
var_bin['badpct']=var_bin['bad_count']/var_bin['sum'].sum()
var_bin['allpct']=var_bin['sum']/var_bin['sum'].sum()
var_bin['cum_badpct']=var_bin['badpct'].cumsum()
var_bin['cum_allpct']=var_bin['allpct'].cumsum()
var_bin['cum_lift']=var_bin['cum_badpct']/var_bin['cum_allpct']

# 第二种Lift计算方式
var_bin['badrate']=var_bin['bad_count']/var_bin['sum']
var_bin['lift']=var_bin['badrate']/df['target'].mean()
```

Lift = 累计坏样本占比/累计总样本占比

Lift = 分箱下的区间坏账率/总样本坏账率

	sum	bad_count	good_count	badpct	allpct	cum_badpct	cum_allpct	cum_lift	badrate	lift
xgb_score										
(205.38899999999998, 448.898]	1000	150	850	0.30	0.1000	0.30	0.1000	3.000000	0.150000	3.000000
(448.898, 483.828]	1000	100	900	0.20	0.1000	0.50	0.2000	2.500000	0.100000	2.000000
(483.828, 508.842]	1000	70	930	0.14	0.1000	0.64	0.3000	2.133333	0.070000	1.400000
(508.842, 530.35]	1001	50	951	0.10	0.1001	0.74	0.4001	1.849538	0.049950	0.999001
(530.35, 550.93]	1001	45	956	0.09	0.1001	0.83	0.5002	1.659336	0.044955	0.899101
(550.93, 570.996]	998	35	963	0.07	0.0998	0.90	0.6000	1.500000	0.035070	0.701403
(570.996, 592.425]	1000	20	980	0.04	0.1000	0.94	0.7000	1.342857	0.020000	0.400000
(592.425, 618.716]	1000	15	985	0.03	0.1000	0.97	0.8000	1.212500	0.015000	0.300000
(618.716, 654.116]	1000	10	990	0.02	0.1000	0.99	0.9000	1.100000	0.010000	0.200000
(654.116, 852.11]	1000	5	995	0.01	0.1000	1.00	1.0000	1.000000	0.005000	0.100000



4. Lift-chart可视化

```
##### PlotLift #####
def PlotLift(preds, labels, n, asc):

    # preds is score: asc=1
    # preds is prob: asc=0

    pred = preds # 预测值
    bad = labels # 取1为bad, 0为good
    lftds = pd.DataFrame({'bad': bad, 'pred': pred})
    lftds['good'] = 1 - lftds.bad

    if asc == 1:
        lftds1 = lftds.sort_values(by=['pred', 'bad'], ascending=[True, True])
    elif asc == 0:
        lftds1 = lftds.sort_values(by=['pred', 'bad'], ascending=[False, True])

    lftds1.index = range(len(lftds1.pred))

    qe = list(np.arange(0, 1, 1.0/n))
    qe.append(1)
    qe = qe[1:]

    lft_index = pd.Series(lftds1.index)
    lft_index = lft_index.quantile(q = qe)
    lft_index = np.ceil(lft_index).astype(int)
    lft_index = list(lft_index)

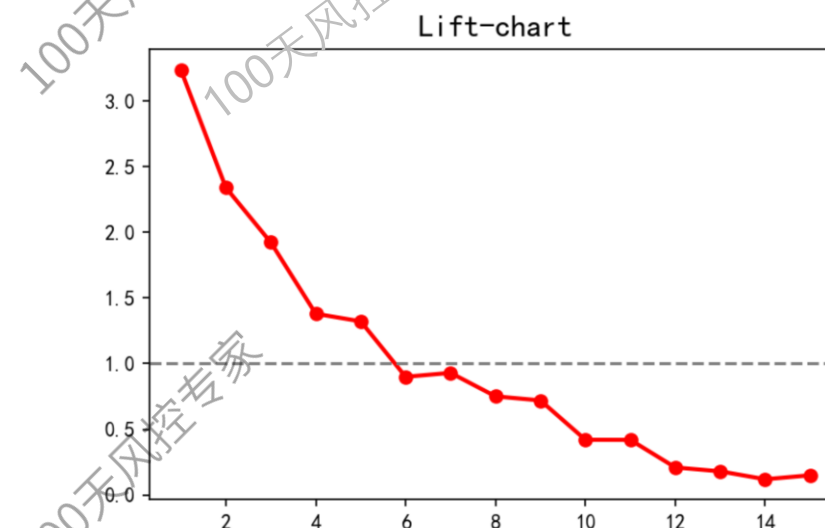
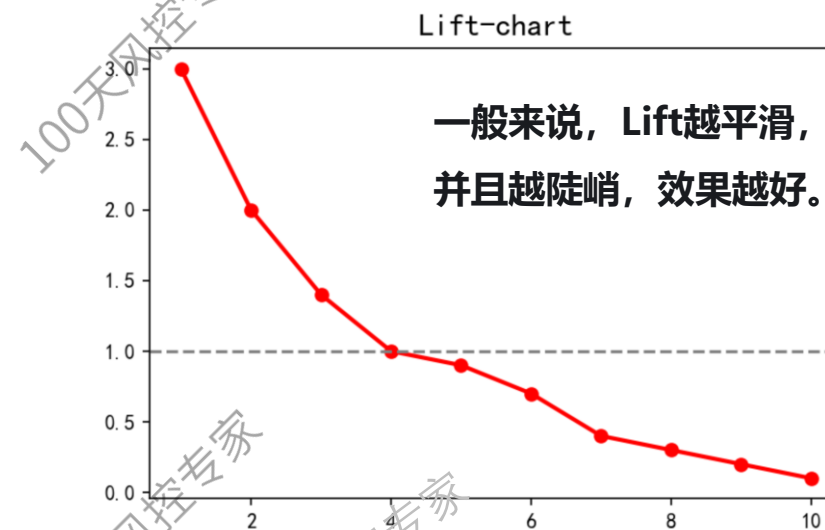
    Rate_all = 1.0*sum(lftds1.bad)/len(lftds1.bad)

    lift = []
    temp = lftds1.loc[lftds1.index<=lft_index[0]]
    lift = lift + [1.0*sum(temp.bad)/len(temp.bad)/Rate_all]

    for i in range(1, len(lft_index)):
        temp = lftds1.loc[(lftds1.index<=lft_index[i]) & (lftds1.index>lft_index[i-1])]
        lift = lift + [1.0*sum(temp.bad)/len(temp.bad)/Rate_all]

    lift_no = range(1, len(lift)+1)

    fig = plt.figure(dpi=150)
    plt.plot(lift_no, lift, 'ro-', linewidth=2)
    plt.axhline(1, color='gray', linestyle='--')
    plt.title('Lift-chart', fontsize=15)
```



扫码进专属群



谢谢

出品人：东哥起飞

解锁风控课程



关注我的公众号



Python数据科学



扫码进专属群

